

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1
За седьмой семестр
По дисциплине «Интеллектуальный анализ данных»
Тема: «РСА»

Выполнил:
Студент 4 курса
Группы ИИ-26
Черноиван А.Н.
Проверил:
Никитин С.К.

Цель работы: научиться применять метод PCA для осуществления визуализации данных

Общее задание

1. Используя выборку по варианту, осуществить проецирование данных на плоскость первых двух и трех главных компонент (двумя способами: 1. вручную через использование `numpy.linalg.eig` для вычисления собственных значений и собственных векторов и 2. с помощью `sklearn.decomposition.PCA` для непосредственного применения метода PCA – два независимых варианта решения);
2. Выполнить визуализацию полученных главных компонент с использованием средств библиотеки `matplotlib`, обозначая экземпляры разных классов с использованием разных цветовых маркеров;
3. Используя собственные значения, рассчитанные на этапе 1, вычислить потери, связанные с преобразованием по методу PCA. Сделать выводы;
4. Оформить отчет по выполненной работе, загрузить исходный код и отчет в соответствующий репозиторий на github.

Задание по вариантам

№ варианта	Выборка	Класс
1	seeds.zip	Последняя колонка

Ход работы

Код программы:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from sklearn.decomposition import PCA
```

```
DATA_PATH = "seeds_dataset.txt"
```

```
COLUMN_NAMES = [
```

```
    "area",
```

```
    "perimeter",
```

```
    "compactness",
```

```
    "kernel_length",
```

```
    "kernel_width",
```

```
    "asymmetry_coefficient",
```

```
"kernel_groove_length",  
"class",  
]
```

```
def prepare_data(path: str = DATA_PATH):
```

```
    df = pd.read_csv(path, sep=r"\s+", header=None, names=COLUMN_NAMES)
```

```
    df = df.fillna(df.mean(numeric_only=True))
```

```
    target = df["class"].to_numpy()
```

```
    data = df.drop(columns=["class"]).to_numpy(dtype=float)
```

```
    return data, target
```

```
def pca_manual(data: np.ndarray, n_components: int):
```

```
    data_centered = data - data.mean(axis=0)
```

```
    cov_matrix = np.cov(data_centered.T)
```

```
    eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
```

```
    sort_index = np.argsort(-1 * eigenvalues)
```

```
    eigenvalues_sorted = eigenvalues[sort_index]
```

```
    eigenvectors_sorted = eigenvectors[:, sort_index]
```

```
    principal_components = eigenvectors_sorted[:, :n_components]
```

```
    projected = np.dot(data_centered, principal_components)
```

```
    return projected.real, eigenvalues_sorted.real
```

```
def explained_loss(eigenvalues_sorted: np.ndarray, n_components: int) -> float:
```

```
    full_info = eigenvalues_sorted.sum()
```

```
reduced_info = eigenvalues_sorted[:n_components].sum()
loss_percent = 100 - reduced_info / full_info * 100
return loss_percent
```

```
def pca_sklearn(data: np.ndarray, n_components: int):
    pca = PCA(n_components=n_components)
    projected = pca.fit_transform(data)
    return projected, pca.explained_variance_ratio_
```

```
def plot_2d(ax, projected_2d, target, title):
    colors = {1: "red", 2: "green", 3: "blue"}
    for cls in np.unique(target):
        mask = target == cls
        ax.scatter(
            projected_2d[mask, 0],
            projected_2d[mask, 1],
            c=colors.get(int(cls), "gray"),
            label=f"class {int(cls)}",
            s=25,
        )
    ax.set_xlabel("PC1")
    ax.set_ylabel("PC2")
    ax.set_title(title)
    ax.legend()
```

```
def plot_3d(ax, projected_3d, target, title):
    colors = {1: "red", 2: "green", 3: "blue"}
    for cls in np.unique(target):
        mask = target == cls
        ax.scatter(
            projected_3d[mask, 0],
```

```

        projected_3d[mask, 1],
        projected_3d[mask, 2],
        c=colors.get(int(cls), "gray"),
        label=f"class {int(cls)}",
        s=25,
    )
ax.set_xlabel("PC1")
ax.set_ylabel("PC2")
ax.set_zlabel("PC3")
ax.set_title(title)
ax.legend()

```

```
def main():
```

```
    data, target = prepare_data()
```

```
    proj_manual_2d, eigvals = pca_manual(data, n_components=2)
```

```
    proj_manual_3d, _ = pca_manual(data, n_components=3)
```

```
    loss_manual_2d = explained_loss(eigvals, 2)
```

```
    loss_manual_3d = explained_loss(eigvals, 3)
```

```
    proj_sklearn_2d, var_ratio_2d = pca_sklearn(data, n_components=2)
```

```
    proj_sklearn_3d, var_ratio_3d = pca_sklearn(data, n_components=3)
```

```
    loss_sklearn_2d = 100 - var_ratio_2d.sum() * 100
```

```
    loss_sklearn_3d = 100 - var_ratio_3d.sum() * 100
```

```
    print(f"Потери (вручную, 2 компоненты): {loss_manual_2d:.2f}%")
```

```
    print(f"Потери (вручную, 3 компоненты): {loss_manual_3d:.2f}%")
```

```
    print(f"Потери (sklearn, 2 компоненты): {loss_sklearn_2d:.2f}%")
```

```
    print(f"Потери (sklearn, 3 компоненты): {loss_sklearn_3d:.2f}%")
```

```
    fig1, axes1 = plt.subplots(1, 2, figsize=(12, 5))
```

```
    plot_2d(axes1[0], proj_manual_2d, target, "PCA вручную (numpy), 2D")
```

```
    plot_2d(axes1[1], proj_sklearn_2d, target, "PCA (sklearn), 2D")

```

```
fig1.tight_layout()
fig1.savefig("pca_2d.png", dpi=150)

fig2 = plt.figure(figsize=(12, 5))
ax_manual = fig2.add_subplot(1, 2, 1, projection="3d")
ax_sklearn = fig2.add_subplot(1, 2, 2, projection="3d")
plot_3d(ax_manual, proj_manual_3d, target, "PCA вручную (numpy), 3D")
plot_3d(ax_sklearn, proj_sklearn_3d, target, "PCA (sklearn), 3D")
fig2.tight_layout()
fig2.savefig("pca_3d.png", dpi=150)

plt.show()

if __name__ == "__main__":
    main()
```

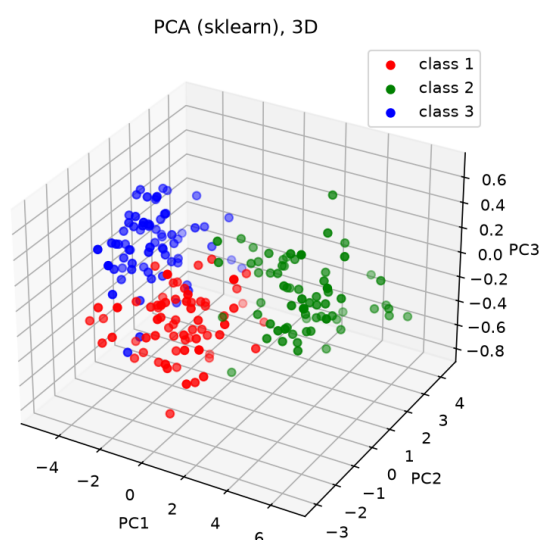
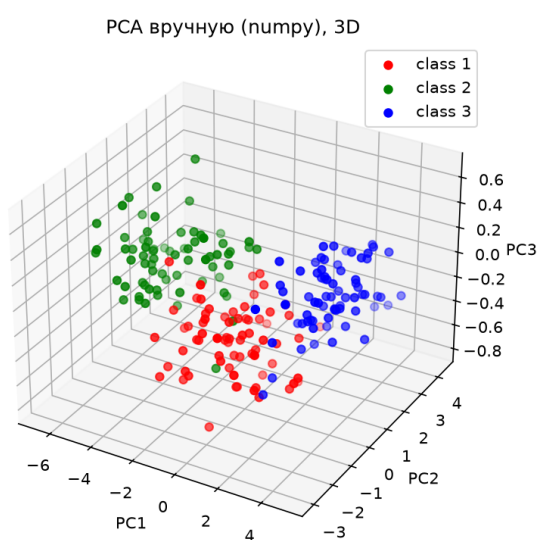
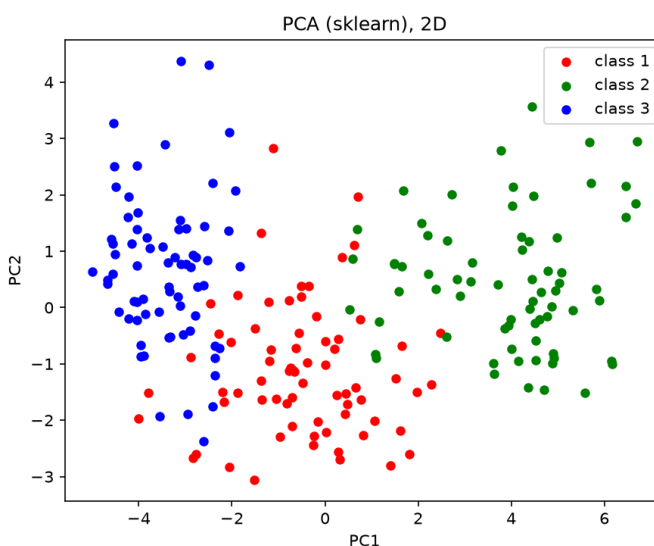
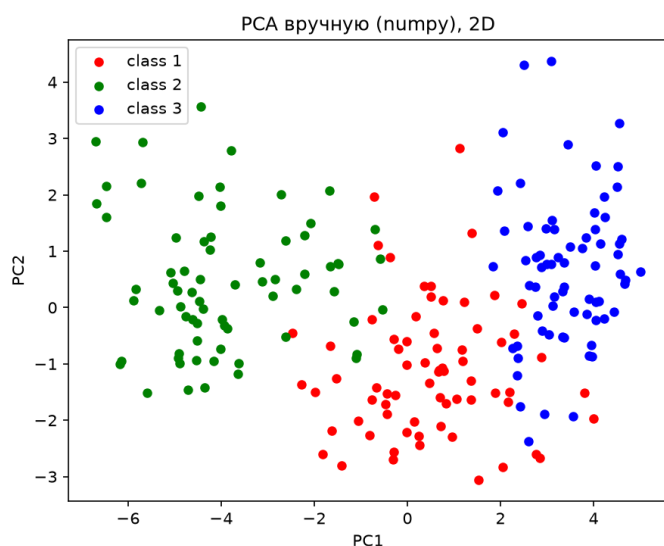
Результаты:

Потери (вручную, 2 компоненты): 0.70%

Потери (вручную, 3 компоненты): 0.13%

Потери (sklearn, 2 компоненты): 0.70%

Потери (sklearn, 3 компоненты): 0.13%



Вывод: Реализация PCA вручную и через sklearn дала идентичные результаты — снижение размерности до 2–3 компонент привело к потерям информативности всего около 0.13–0.70%, при этом классы зерна на графиках визуально хорошо разделяются, что подтверждает эффективность метода для сжатия и визуализации данных.